

AWS re:Invent

NOV. 28 – DEC. 2, 2022 | LAS VEGAS, NV

CMP323

Demystifying Amazon EC2 Provisioning

Sudhi Bhat (he/him)

Sr. Specialist Solutions Architect, Flexible Compute
Amazon Web Services

Peter Manastyrny (he/him)

Sr. Product Manager, EC2 Core
Amazon Web Services



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Agenda

- Amazon Elastic Cloud Compute (Amazon EC2)
- Understanding EC2 instance provisioning
- Key considerations while provisioning EC2 capacity
- Use cases

Broadest and deepest compute platform choice

CATEGORIES

General purpose
Burstable
Compute intensive
Memory intensive
Storage (High I/O)
Dense storage
GPU compute
Graphics intensive



CAPABILITIES

Choice of processor
(AWS, Intel, AMD)
Fast processors
(up to 4.0 GHz)
High memory footprint
(up to 12 TiB)
Instance storage
(HDD, SSD, NVMe)
Accelerated computing
(GPUs and FPGA)
Networking
(up to 100 Gbps)
Bare Metal
Size
(Nano to 32xlarge)



OPTIONS

Amazon EBS
Amazon Elastic Inference



MORE THAN
550+
INSTANCE TYPES
for virtually every
workload and
business need

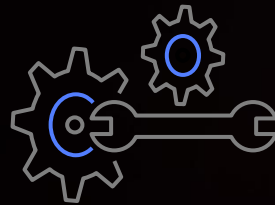
Tools for managing cost and complexity



Easy to choose

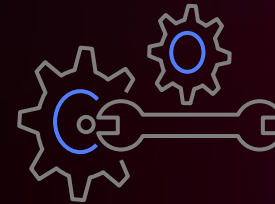
EC2 Instance Discovery

AWS Compute
Optimizer



Easy to build

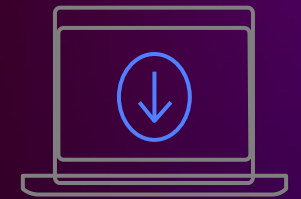
EC2 Image Builder



Easy to provision

CLI/Console/SDK

AWS Managed Services
(AMS)



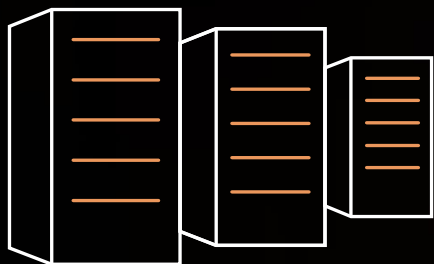
Easy to save

AWS Savings Plans

Amazon EC2 Spot
Instances

Amazon EC2

- VIRTUAL SERVERS IN THE CLOUD



Physical servers in
AWS global regions

550+ instance types

Guest 1

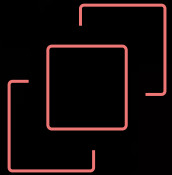
Guest 2

Guest n

Hypervisor

Host server

Options for Launching EC2 Capacity



RunInstances

Launch the specified number of instances using an AMI



EC2 Fleet "instant"

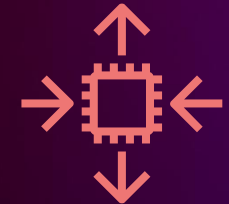
Replaces RunInstances, No Fleet Management

recommended



Spot/EC2 Fleet "maintain"

Launch massive fleets or have fine-grained placement control



EC2 Auto Scaling groups

Best fit for the majority of use cases. Higher level of abstraction.

recommended

Instance Management

RunInstances

API allows you to launch one or more instances

- No Instance Life Cycle Management
- No options to use multiple instance types
- **Recommendation**
 - **EC2 Fleet “Instant” mode instead**

API

```
https://ec2.amazonaws.com/?Action=RunInstances &ImageId=ami-31814f58  
&InstanceType=m1.small  
&MaxCount=1  
&MinCount=1  
&KeyName=my-key-pair  
&SubnetId=subnet-b2a249da  
&AUTHPARAMS
```

AWS CLI

```
aws ec2 run-instances \ --image-id ami-0abcdef1234567890 \ --instance-type t2.micro \ --key-name MyKeyPair
```


Spot Fleet

Collection, or fleet, of Spot Instances, and optionally On-Demand Instances

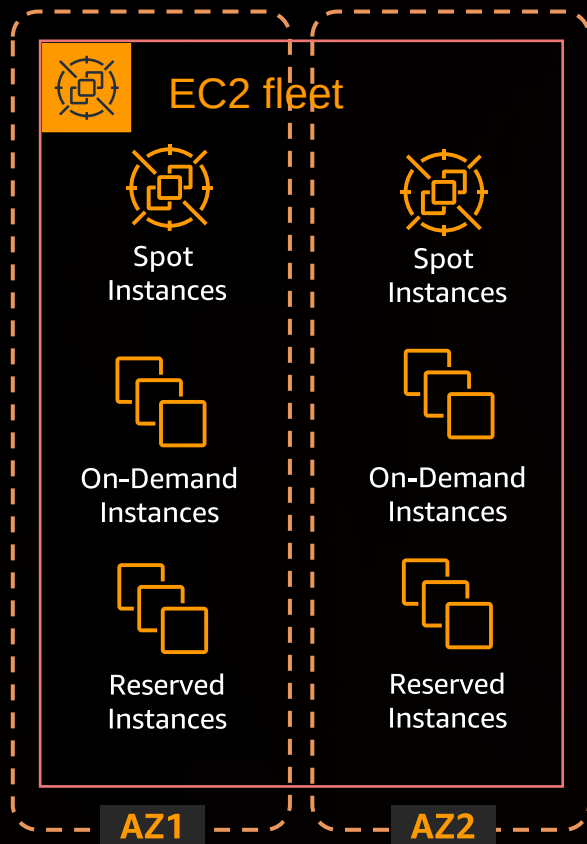
- Types of Spot Fleet requests
 - “request”
 - “maintain”
- API and Console Support
- DO NOT USE:
 - **RequestSpotFleet API**
 - **RequestSpotInstances**
- **Recommendation:**
 - **Use EC2 Auto Scaling Groups when instance life cycle management is required**
 - **Else, Use EC2 Fleet API**

```
{
  "SpotFleetRequestConfig": {
    "AllocationStrategy": "lowestPrice",
    "ExcessCapacityTerminationPolicy": "default",
    "IamFleetRole": "arn:aws:iam::111122223333:role/role",
    "LaunchSpecifications": [
      {
        "ImageId": "ami-0123456789EXAMPLE",
        "InstanceType": "c4.large",
        "TagSpecifications": [
          {
            "ResourceType": "instance",
            "Tags": [
              {
                "Key": "Cost-Center",
                "Value": "123"
              }
            ]
          }
        ]
      }
    ]
  },
  "SpotPrice": "5",
  "TargetCapacity": 2,
  "TerminateInstancesWithExpiration": true,
  "Type": "maintain",
  "ReplaceUnhealthyInstances": true,
  "InstanceInterruptionBehavior": "terminate",
  "InstancePoolsToUseCount": 1
}
```

aws ec2 request-spot-fleet --spot-fleet-request-config file://config.json

Amazon EC2 Fleet

*Allows to **synchronously** provision capacity across different instance types, Availability Zones (AZ), and purchase options with a single API*



"Builders API"

Use all three purchase options to optimize costs

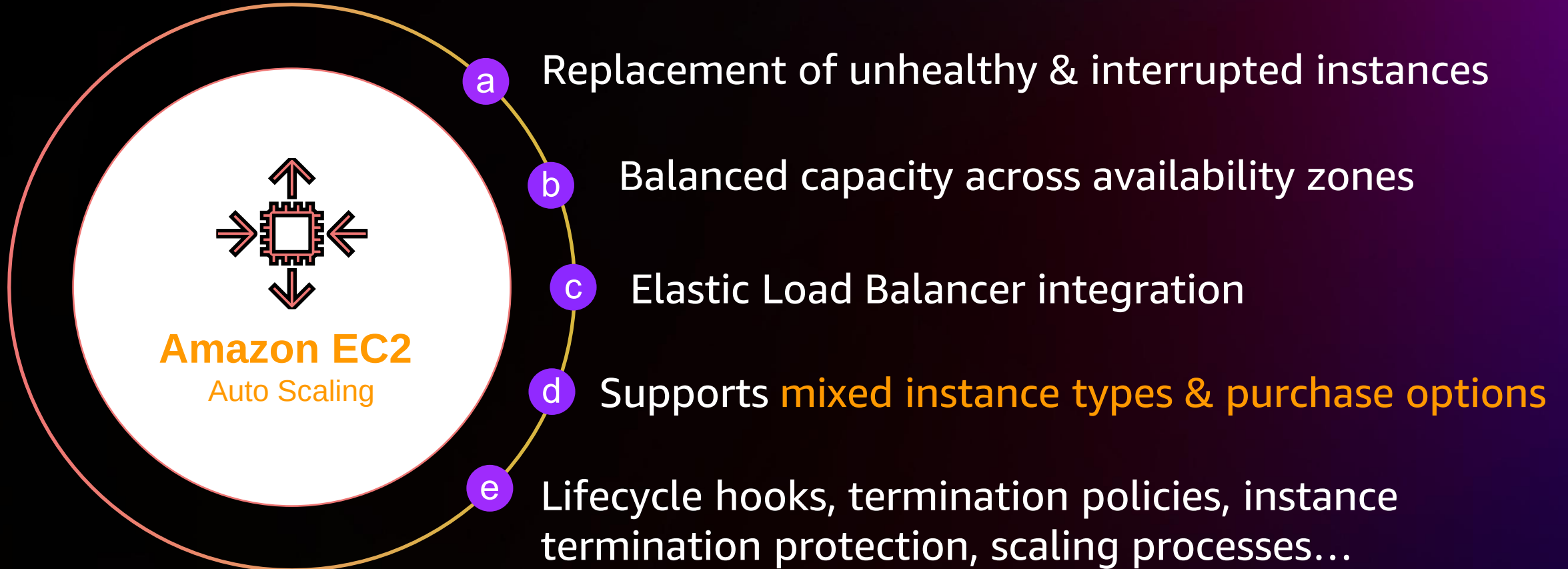
Benefits

- Reduce costs
- Increase operational efficiency
- Reduce development effort

Key features

- Flexible capacity allocation
- Massive scale
- Simplified provisioning
- Instant Fleets: Drop-In replacement for RunInstances

Amazon EC2 Auto Scaling groups



So, which provisioning mechanism should I use?

Use EC2 Fleet for DIY control over instance management, otherwise let EC2 Auto Scaling Groups reduce the undifferentiated heavy lifting.

Are there any other considerations while Provisioning EC2 Capacity?

Amazon EC2 Instance pools explained

C4	AZ1	AZ2	AZ3	On-Demand
8XL	\$0.28	\$0.27	\$0.29	\$1.76
4XL	\$0.21	\$0.19	\$0.16	\$0.88
2XL	\$0.08	\$0.07	\$0.08	\$0.44
XL	\$0.04	\$0.05	\$0.04	\$0.22
L	\$0.01	\$0.01	\$0.02	\$0.11

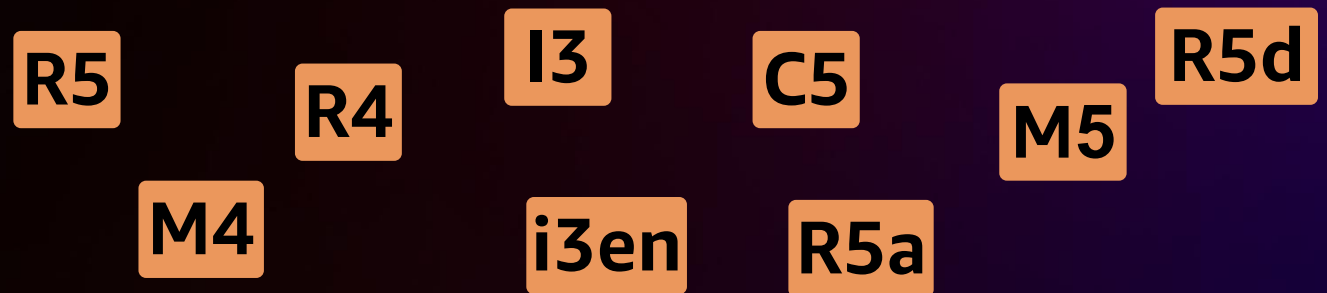
Each instance family

Each instance size

Each availability zone

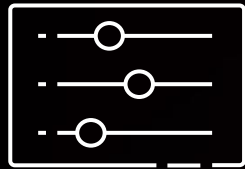
In every Region

Is a separate **pool**



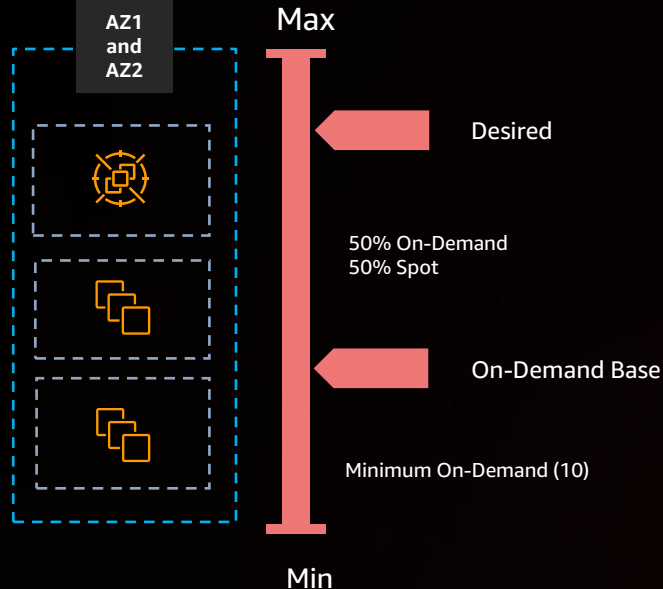
Flexibility is key to successful adoption

Instance
flexible



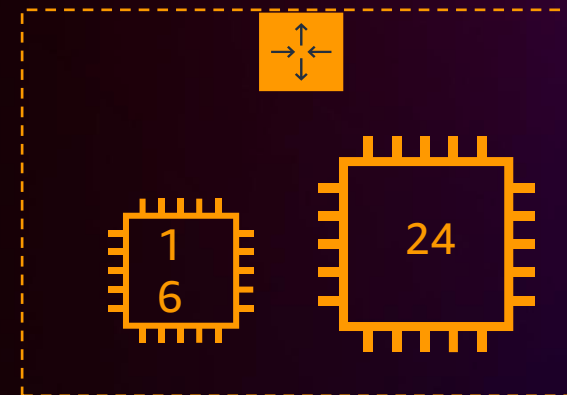
- More than one instance type can get the job done
- Instance weighting gives you more flexibility on instance types
- Multiple instance types are key to resilient clusters

Mixed Instance &
Purchase Options



Instance Weights

TargetCapacity: 40



Let Amazon EC2 Auto Scaling do the heavy lifting

```
1 AutoScalingGroupName: myAutoScalingGroup
2 CapacityRebalance: true
3 DesiredCapacity: 12
4 MaxSize: 15
5 MinSize: 12
6 MixedInstancesPolicy:
7   InstancesDistribution:
8     OnDemandBaseCapacity: 3
9     OnDemandPercentageAboveBaseCapacity: 25
10    SpotAllocationStrategy: capacity-optimized
11  LaunchTemplate:
12    LaunchTemplateSpecification:
13      LaunchTemplateName: myLaunchTemplate
14      Version: $Default
15    Overrides:
16      - InstanceType: c5.large
17      - InstanceType: c5a.large
18      - InstanceType: m5.large
19      - InstanceType: m5a.large
20      - InstanceType: c4.large
21      - InstanceType: m4.large
22      - InstanceType: c3.large
23      - InstanceType: m3.large
24 VPCZoneIdentifier: mySubnet1,mySubnet2,mySubnet3
```

5. Allow EC2 Auto Scaling to proactively rebalance Spot Instances with elevated risk of interruption to increase application availability

1. Allow EC2 Auto Scaling to provision a combination of On-Demand Instances and Spot Instances across multiple instance types

3. Set a base capacity for On-Demand Instances covered by a Savings Plan

4. Use the capacity-optimized allocation strategy to find the optimal spare capacity to reduce cost of interruptions

2. Provide a flexible list of instance types and Availability Zones to allow EC2 Auto Scaling to optimize for performance, cost, and availability

Learn more: <https://amzn.to/3lhBPvI>

Use the right Amazon EC2 allocation strategy



On-demand allocation

- Prioritized
- Lowest-price (default for mixed instance groups)



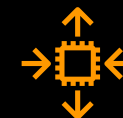
Spot allocation

- Price-capacity-optimized (Recommended)
- Capacity-optimized
- Lowest-price

New!



EC2 Fleet



Amazon EC2
Auto Scaling



On-demand instances



Spot instances

How do I take advantage of newer EC2 instances?



Attribute Based Instance Selection

Provision capacity based on instance attributes depending on compute requirements

```
{
  "LaunchTemplateConfigs": [
    {
      "LaunchTemplateSpecification": {
        "LaunchTemplateName": "My-launch-template",
        "Version": "1"
      },
      "Overrides": [
        {
          "InstanceRequirements": {
            "VCpuCount": {
              "Min": 2,
              "Max": 8
            },
            "MemoryMib": {
              "Min": 0,
              "Max": 10240
            },
            "MemoryGiBPerVCpu": {
              "Max": 10000
            },
            "RequireHibernateSupport": true
          }
        }
      ]
    }
  ],
  "TargetCapacitySpecification": {
    "TotalTargetCapacity": 5000,
    "DefaultTargetCapacityType": "spot",
    "TargetCapacityUnitType": "vcpu"
  }
}
```

AWS CLI

Instance type requirements [Info](#)

[Reset to launch template](#)

You can keep the same instance attributes or instance type from your launch template, or you can choose to override the launch template by specifying different instance attributes or manually adding instance types.

☒ Specify instance attributes

Provide your compute requirements. We fulfill your desired capacity with matching instance types based on your allocation strategy selection.

☐ Manually add instance types

Add one or more instance types. Any of the instance types may be launched to fulfill your desired capacity based on your allocation strategy selection.

Required instance attributes

Enter your compute requirements in virtual CPUs (vCPUs) and memory.

vCPUs

Enter the minimum and maximum number of vCPUs per instance.

minimum maximum

☐ No minimum

☐ No maximum

Memory (GiB)

Enter the minimum and maximum GiBs of memory per instance.

minimum maximum

☐ No minimum

☐ No maximum

Additional instance attributes - *optional*

Add instance attributes to further limit which instance types may be used to fulfill your desired capacity.

Network bandwidth (Gbps) minimum maximum [×](#)

Choose attribute ▼

[Add attribute](#)

► Preview matching instance types (77)

This list includes all the instance types that match your compute requirements. Amazon EC2 may provision from any of these instance types. The exact instance types that are used to fulfill your desired capacity depend on the allocation strategy you choose and available capacity.

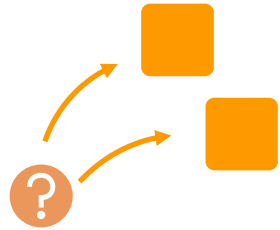


How do I control my Compute cost?

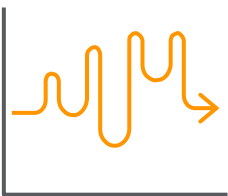


Cost Considerations: Amazon EC2 Purchase Options

On-Demand



Pay for compute capacity by **the second** with no long-term commitments

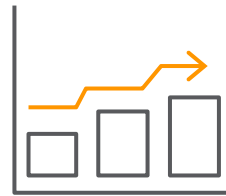


Spiky **stateful** workloads, to define needs

Reserved Instances / Savings Plan



Make a 1 or 3 year commitment and receive a **significant discount** off On-Demand prices



Committed and steady-state usage

EC2 Spot



Spare Amazon EC2 capacity at **savings of up to 90%** off On-Demand prices



Fault-tolerant, flexible, **stateless** workloads

Let's review some use cases?



I want to automate the lifecycle management through a configurable API, which provisioning mechanism should I use?

I use Spot Fleet Today, should I start using Auto Scaling Groups instead?

I want to reduce the time it takes to scale my application and decrease latency

Examples:

- **Sudden upturn in Traffic**
- **Application with a longer boot time..**

I got it, but who uses EC2 Fleet Today?



Have you heard of Karpenter?

Learn more at <https://karpenter.sh/>

Breakout Session:

Wednesday, November 30

Topic: Modern applications: An infrastructure perspective

Location: Level 1 North, South Pacific F, Mandalay Bay



I want to launch X vCPUs, I have a few qualified instance types that fit my workload, but for a specific job they should be identical, and in the same AZ. Also, don't launch anything if you can't get me all the capacity I asked for.

Examples

- Tightly couples HPC usecases, which is AZ flexible.
- Any identical group of instance types with EFA support

I have a tool or platform that launches single-instance test environments for developers every day from the latest AMI, currently using RunInstances

Hudl Scales Video Processing and Boosts Reliability by Optimizing on Amazon EC2 Spot Instances



Using Amazon EC2 Auto Scaling, we gain proportional scaling that's managed by AWS, whereas previously we would have had to build fairly complex in-house scaling algorithms to accomplish the same thing."

Jordan Degner

Director of Engineering for Core Video, Hudl

Summary

- **DO NOT USE RequestSpotFleet, RequestSpotInstances**
- Upgrade from RunInstances to EC2 Fleet ("instant") or EC2 Auto Scaling Groups
- Considerations: Instance Flexibility, Allocation Strategies, Purchasing Options, Availability
- Amazon EC2 Auto Scaling for provisioning Amazon EC2 helps with,
 - Better fault tolerance
 - Better availability
 - Better cost management

Rule of Thumb: Use EC2 Fleet for DIY control over instance management, otherwise let Auto Scaling Groups reduce the undifferentiated heavy lifting

Thank you!

Sudhi Bhat

sudheenb@amazon.com

Peter Manastyrny

pmanasty@amazon.com



Please complete the session survey in the **mobile app**



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.